

Learning Plane — the part that gets better over time

Learning Plane — the part that gets better over time

The other two planes handle *now*. The learning plane handles *next week*. Nothing here runs while a user waits — it's all offline, batch, asynchronous. That's why it can afford to be expensive.

What problem it solves

You picked `pii_threshold: 0.5` and `low_band: 0.3`. Where did those numbers come from? Right now: someone's guess.

The uncomfortable question a reviewer will ask is "**how do you know your system is working?**" You can count how many things you blocked. You cannot, from that alone, count how many bad things you let through — because if you'd detected them you'd have blocked them. Your false negatives are invisible by construction.

The learning plane exists to make them visible, and then to turn what you learn into better thresholds.

Four components

1. Audit ledger

Every decision, recorded immutably. Serves three different audiences from one table:

- **Compliance** — "why was this blocked in March" → look up the record, read the `policy_hash`, pull that exact bundle
- **Finance** — cost per interaction, per tier, per use case
- **Engineering** — the labelled data everything else here depends on

Hash-chaining means each record includes a hash of the previous record.

Tamper with record 4,000 and every hash after it breaks. That's how you prove to an auditor that nobody quietly edited the log after an incident. Implementation is one line:

```
record["prev_hash"] = last_hash
record["hash"] = sha256(json.dumps(record, sort_keys=True))
```

The critical design rule, repeated because it matters: **types and scores, never values**. You record that a credit card was detected at score 0.98 at characters 12–31. You never record the number. Your system inspects every prompt in the enterprise; if the ledger held raw content it would be the highest-value breach target in the company.

2. Reviewer queue

"Flag for review" has to land somewhere. This is where a human looks at flagged cases and marks them correct or incorrect.

Two reasons it matters:

- It's your **only source of ground truth on false positives**. A human saying "that wasn't actually PII" is the label.
- It's your **human-accountability story**. The brief asks when a person gets pulled in; this is the answer, with a name and a timestamp attached.

Design point worth mentioning: prioritise the queue by uncertainty, not volume. Reviewing cases where the system scored 0.51 teaches you far more than reviewing ones it scored 0.99.

3. Shadow eval — the false-negative estimator

This is the sharpest idea in your architecture, so make sure the team understands it.

Sample ~2% of the traffic that your cascade **allowed**, and run the expensive T2 judge on it anyway — after the fact, offline, with the user long gone. Then count how often the judge disagrees.

That disagreement rate, scaled up, is your **estimated false-negative rate**. You've measured the thing that's normally unmeasurable, at 2% of the cost of measuring it properly.

2% sample of ALLOWED traffic → run T2 offline → judge says "actually risky"
→ 6 of 400 sampled → est. miss rate $\approx$ 1.5%

Two things to be careful about, because a sharp reviewer will probe:

- The judge is not ground truth either. It's a *better* detector than T1, not a perfect one. So this is an estimate with its own error bars — say so rather than presenting it as fact.
- Sample randomly, not from the interesting-looking traffic. Biased sampling gives you a biased miss rate.

Don't confuse this with shadow deploy (in the control plane). Shadow deploy compares two *policy versions*. Shadow eval estimates what your *current* policy is missing. Similar names, different jobs — the team will mix these up at least once.

4. Calibration

Takes everything above — reviewer labels, shadow eval results, cost and latency telemetry — and answers: *what should the thresholds actually be?*

The mechanism doesn't need to be clever. Sweep candidate values over your labelled set and plot the resulting false-positive rate, false-negative rate, escalation rate and cost. That gives you the tradeoff curve, and a human picks a point on it based on risk appetite.

That's the honest framing, and it's better than claiming automated optimisation: **calibration doesn't decide, it shows the tradeoff**. The brief says most real systems "deliberately tune this tradeoff rather than solve it away" — this component is you doing exactly that, visibly.

The word "calibration" also has a technical meaning worth knowing: a detector is *calibrated* if things it scores 0.7 are actually risky about 70% of the time. Most classifiers are badly calibrated out of the box. Reliability diagrams and Platt scaling are the standard fixes. Mention it if asked; don't build it.

The loop that closes

This is the part the original diagram was missing, and it's what makes it a learning plane rather than a logging plane:

```
calibration proposes: low_band 0.30 → 0.35
  ↓
  human approves ← the gate. never automatic.
    ↓
  control plane: new policy version, new hash
    ↓
  shadow deploy: run v7 vs v8 side by side before enforcing
    ↓
  data plane enforces v8
    ↓
  new decisions flow back to the ledger
```

Why the human gate is non-negotiable: without it, a feedback loop can drift. Reviewers get fatigued and start rubber-stamping "not actually a problem," calibration reads that as over-flagging, thresholds loosen, fewer things get flagged, fewer get reviewed, and the system quietly relaxes itself into uselessness. A human approving each version change breaks that loop and gives you an accountable owner per version.

Say this in the pitch: *"our guardrail's own configuration changes are themselves versioned, human-approved and audited."* Governing the governor is a question most teams won't have anticipated.

Metrics you report to a skeptical stakeholder

The brief asks for this explicitly, so have the list ready:

Metric	Source
False positive rate	reviewer queue
Estimated false negative rate	shadow eval
Escalation rate (% hitting T2)	ledger
p50 / p95 added latency	ledger

Metric	Source
Cost per thousand checked interactions	ledger
Override rate per use case	reviewer queue
Cost avoided (cache, routing, cutoffs)	ledger

The last row is the one that closes your business case. If you can end on *"governance added ₹X per thousand interactions and cost controls saved ₹3X,"* you've paid for the system with your own telemetry rather than a projection.

Build plan

Real: the ledger with hash-chaining (it's genuinely simple and demos beautifully — tamper with a record on stage and show the chain break), and a calibration sweep script producing the tradeoff curve.

Stubbed: shadow eval as an offline script over a fixed labelled set of ~200 stored interactions rather than live sampling. Reviewer queue as a small table in the console with approve/reject buttons, or even a CSV.

The tradeoff curve from calibration is the single highest-value artifact in this plane. One chart, and it answers the over-flagging question, the metrics question, and the "how do you know it works" question at once.